REPORT 4

Repository URL: https://github.com/anakhaskartha/LED.git

1. Code Explanation

1.1 Header File (led.h)

The header file acts as the public interface, defining structures, constants, and function prototypes:

- **Include Guards:** #ifndef LED_H and #define LED_H prevent multiple inclusions during preprocessing and avoid redefinition conflicts.
- **State Macros:** #define OFF 0 and #define ON 1 establish readable binary state definitions for LED status.
- **Structure Definition:** struct LED encapsulates an integer led_no to identify the LED and an integer pointer *status pointing to its active state value.
- **Function Prototypes:** Lists the names of the functions so the rest of the program knows they exist. (initialise, set, reset, toggle, display, and deallocate.)

1.2 Implementation File (led.c)

This file implements the functions declared in led.h to manage dynamic memory allocation, state mutation, and deallocation:

- initialise(int n):
  - Allocates a contiguous memory block on the heap sized for n struct LED elements using malloc(n*sizeof(struct LED)).

- Iterates from 0 to n - 1 to assign led_no = i + 1, allocates individual memory for each status pointer using malloc(sizeof(int)), and defaults the value to OFF.
- Returns the pointer to the allocated struct LED array.

- set(struct LED *led):
  - Dereferences the status pointer member (*(led->status)) and sets its value to ON.
- reset(struct LED *led):
  - Dereferences the status pointer member (*(led->status)) and sets its value to OFF.
- toggle(struct LED *led):
  - Evaluates *(led->status): if equal to ON, updates it to OFF; otherwise, updates it to ON.
- display(struct LED *leds, int n):
  - Loops across the array and prints fomatted output indicating whether each LED is [LED %d: ON] or [LED %d: OFF] based on the value held at *(leds[i].status).
- deallocate(struct LED *leds, int n):
  - Prevents memory leaks by first freeing each dynamically allocated status pointer inside the loop (free(leds[i].status)).
  - Frees the base array pointer (free(leds)) after all nested allocations have been released.

1.3 Driver Program (main.c)

The driver coordinates execution flow and user input:

- **User Input:** Reads the number of LEDs (n) and toggle cycles (m) using scanf.
- **Dynamic Setup:** Calls initialise(n) to construct the array on the heap.

- **Alternating Pattern Configuration:** Iterates through indices 0 to n - 1. Even indices (i % 2 == 0) are assigned reset(&leds[i]), while odd indices are assigned set(&leds[i]).
- **Execution & Display:** Displays the starting state using display(leds, n), then runs an outer loop for m cycles. In each cycle, it toggles every LED via toggle(&leds[i]) and prints the resulting states with display(leds, n).
- **Cleanup:** Calls deallocate(leds, n) to release heap memory before program termination.